

UNIVERSIDAD SIGLO 21

Carrera: Ingeniería de Software

Trabajo Final de Grado — Segunda Entrega (Entregable Completo)

FreelanceControl: sistema de gestión financiera para monotributistas argentinos

Alumno: Marcos Joaquin

Profesor TFG: Mainero, Alejandro

Año académico: 2026

Título

FreelanceControl: sistema de gestión financiera para monotributistas argentinos.

Introducción

El trabajo independiente bajo el régimen de Monotributo en Argentina presenta desafíos de gestión financiera que las herramientas contables convencionales no logran cubrir. Los monotributistas operan con flujos de caja variables, límites anuales de facturación que condicionan su categoría fiscal y una alta dispersión documental entre planillas, comprobantes y aplicaciones genéricas.

El presente trabajo final de grado documenta el desarrollo de FreelanceControl, un sistema web de gestión financiera predictiva orientado a este segmento. La solución integra un módulo de clasificación de gastos basado en procesamiento de lenguaje natural con algoritmos Naive Bayes y Support Vector Machines entrenados localmente, un motor de predicción de ingresos mediante el modelo Prophet, un módulo fiscal con monitoreo automatizado de la categoría de Monotributo y un componente de auditoría que detecta inconsistencias en los registros financieros del usuario.

Adicionalmente, el sistema dispone de un módulo de importación masiva multiformato que permite consolidar movimientos financieros previamente registrados por el usuario en archivos CSV y Excel exportados desde su entidad bancaria, reduciendo la carga manual de datos.

La arquitectura se construye sobre FastAPI, PostgreSQL y React, con un núcleo de procesamiento local que garantiza soberanía sobre la información financiera del usuario: el sistema no transmite texto libre ni datos identificables a servicios externos, restringiendo el uso de modelos de lenguaje a la generación de resúmenes y recomendaciones a partir de información numérica agregada.

Antecedentes

El control financiero de los trabajadores independientes evolucionó a través de tres etapas tecnológicas diferenciadas. La primera, predominante hasta finales de la década de 1990, se caracterizó por registros manuales en cuadernos contables y la delegación de la gestión tributaria a contadores externos. Los movimientos se asentaban en libros físicos con periodicidad variable, sin posibilidad de consolidación automática ni proyección de escenarios futuros.

La segunda etapa, impulsada por la masificación de las computadoras personales y las suites ofimáticas, trasladó el registro a hojas de cálculo locales como Microsoft Excel y posteriormente Google Sheets. Esta transición incorporó capacidad de cómputo y fórmulas, pero mantuvo la dependencia del registro manual y la responsabilidad del usuario sobre la integridad de los datos.

La tercera etapa, vigente desde mediados de la década de 2010, introdujo plataformas web especializadas en gestión contable y de facturación. Soluciones internacionales como QuickBooks y FreshBooks, junto con desarrollos locales como Contabilium, Xubio y Colppy, ofrecen funcionalidades de facturación electrónica, integración con organismos fiscales y reportes básicos.

Estas plataformas, sin embargo, fueron diseñadas con foco en pequeñas y medianas empresas con estructura administrativa formal, no en operadores individuales bajo regímenes tributarios simplificados.

La incorporación de inteligencia artificial al dominio financiero comenzó hacia 2020, principalmente mediante la integración con modelos de lenguaje de gran escala provistos por terceros, lo que introdujo dependencia tecnológica y exposición de datos sensibles a infraestructura externa. La evolución reciente del campo se orienta hacia modelos de aprendizaje automático

entrenados localmente, alineados con principios de soberanía de datos y reducción de costos operativos por consumo de servicios externos.

Descripción del área problemática

El segmento de trabajadores independientes inscriptos en el régimen de Monotributo en Argentina enfrenta cuatro problemáticas concretas. La primera es la variabilidad estructural del flujo de caja. Los ingresos del monotributista dependen de la finalización de entregables, los plazos de pago de cada cliente y las condiciones contractuales pactadas por proyecto, lo que produce dispersión mensual y dificulta la planificación financiera. Las herramientas convencionales registran movimientos pero no proyectan escenarios futuros con base en el historial propio del usuario, dejando la anticipación financiera como una tarea cognitiva delegada al criterio individual.

La segunda problemática se vincula al control de la categoría fiscal. El régimen de Monotributo establece categorías diferenciadas para la prestación de servicios, cada una con un límite anual de facturación que se actualiza periódicamente conforme a la normativa vigente publicada por la Administración Federal de Ingresos Públicos. Superar el límite vigente fuerza la recategorización obligatoria o, en casos extremos, la exclusión del régimen. El monitoreo manual de este indicador exige al freelancer consolidar la facturación acumulada y compararla periódicamente contra la tabla oficial, tarea propensa a omisiones que derivan en sanciones fiscales.

La tercera problemática consiste en la ausencia de mecanismos automatizados de auditoría sobre los registros financieros. El monotributista no dispone de herramientas que detecten gastos cargados por duplicado, anomalías estadísticas en patrones de gasto por categoría, facturas

vencidas sin cobrar o pagos faltantes de la cuota mensual de Monotributo. La detección de estas inconsistencias depende del cruce manual entre comprobantes, extractos bancarios y planillas, proceso que se posterga sistemáticamente hasta el cierre fiscal anual.

La cuarta problemática es la fragmentación documental. La información financiera del freelancer se distribuye entre planillas locales, aplicaciones móviles, correos electrónicos con comprobantes adjuntos, archivos PDF de facturas emitidas y exportaciones CSV de homebanking. Esta dispersión imposibilita la consulta consolidada y obliga a procesos manuales de reconciliación.

Las tendencias del mercado de software para trabajadores independientes muestran un crecimiento anual sostenido, con migración hacia soluciones que integren inteligencia artificial para clasificación automática de movimientos y predicción de indicadores financieros, segmento donde la oferta local resulta limitada.

Justificación

El sistema satisface la necesidad de centralizar el registro de ingresos, gastos, facturas, obligaciones fiscales y proyecciones financieras del monotributista en una plataforma única, accesible desde cualquier dispositivo con conexión a internet. La fragmentación actual entre planillas, aplicaciones móviles y correo electrónico genera duplicación de datos, omisiones y errores de cálculo que pueden derivar en sanciones fiscales no previstas. FreelanceControl resuelve esta dispersión mediante un modelo de datos relacional unificado que vincula transacciones, facturas, alertas de auditoría y proyecciones bajo el dominio único del usuario autenticado.

La innovación tecnológica reside en la implementación de un pipeline de operaciones de aprendizaje automático ejecutado íntegramente sobre infraestructura local. El clasificador de

gastos utiliza vectorización TF-IDF combinada con Naive Bayes Multinomial para datasets pequeños y Support Vector Machines lineales para volúmenes superiores a cien ejemplos, ambos provistos por la biblioteca Scikit-learn.

El motor predictivo emplea Prophet, modelo aditivo desarrollado por Meta que descompone series temporales en tendencia, estacionalidad y ruido, generando proyecciones con intervalos de confianza configurables, y se complementa con una estrategia de arranque en frío basada en media móvil para usuarios con historial financiero reducido. La elección de FastAPI como framework de backend responde a su rendimiento asíncrono basado en ASGI y a la generación automática de documentación OpenAPI. PostgreSQL aporta integridad referencial mediante restricciones de clave foránea y soporte nativo para tipos numéricos de precisión arbitraria, requisito para operaciones financieras.

El valor diferencial para el usuario radica en la soberanía de los datos. FreelanceControl entrena y ejecuta su clasificador de gastos dentro del mismo entorno donde reside la base de datos, sin transmitir descripciones de gastos, montos individuales ni información identificable a servicios externos. La detección de la estructura de los archivos importados se resuelve mediante heurísticas locales construidas a partir del relevamiento de los formatos de exportación de las principales entidades bancarias argentinas, eliminando la necesidad de delegar esta tarea a modelos de lenguaje externos.

El uso de servicios de inteligencia artificial provistos por terceros queda restringido a dos funciones específicas: la generación del resumen financiero mensual en lenguaje natural y la generación de recomendaciones personalizadas. En ambos casos, el sistema transmite únicamente información numérica agregada (totales por categoría, conteos de movimientos, porcentajes de

variación y montos consolidados), sin enviar texto libre, descripciones individuales ni datos identificables del usuario.

Esta arquitectura preserva el control efectivo del freelancer sobre su información financiera, reduce los costos operativos asociados al consumo de APIs comerciales y simplifica la auditoría de la política de privacidad, dado que la verificación se reduce a inspeccionar los dos puntos del código donde se convoca al servicio externo.

Objetivo General del Proyecto

Desarrollar un sistema web de gestión financiera para monotributistas argentinos, basado en una arquitectura de servicios REST con FastAPI y persistencia en PostgreSQL. El que debe integrar un clasificador de gastos por procesamiento de lenguaje natural entrenado localmente con precisión igual o superior al 70% sobre 12 categorías predefinidas. Incluye un motor de predicción de ingresos a seis meses mediante Prophet, un módulo fiscal de monitoreo de Monotributo y un componente de auditoría automatizada, dentro de un plazo de seis meses desde el inicio del desarrollo.

Objetivos Específicos del Proyecto

- Construir un módulo de gestión transaccional que consolide el registro de ingresos, gastos y facturas con máquina de estados, soportado por un modelo de datos relacional con aislamiento por usuario autenticado, validación automatizada de esquemas y un mecanismo de importación masiva multiformato desde archivos CSV y Excel con detección local de la estructura del archivo, validación previa de extensión y tamaño

máximo de diez megabytes, vista preliminar de los registros y persistencia transaccional con capacidad de reversión ante errores.

- Diseñar un motor de clasificación de gastos basado en procesamiento de lenguaje natural, con algoritmos Naive Bayes y Support Vector Machines sobre vectorización TF-IDF, con precisión igual o superior al 70% sobre 12 categorías predefinidas medida mediante validación cruzada de cinco particiones, capaz de aprender de las correcciones del usuario mediante reentrenamiento automático y de operar sin dependencia de servicios externos.
- Implementar un módulo fiscal y de auditoría automatizada que monitoree el porcentaje de utilización del límite anual de la categoría de Monotributo del usuario, detecte gastos duplicados por ventana temporal e identifique anomalías estadísticas mediante puntaje z.
- Integrar un pipeline de operaciones de aprendizaje automático que genere proyecciones de ingresos a seis meses con intervalos de confianza mediante el modelo Prophet, incorporando una estrategia de arranque en frío basada en media móvil para usuarios con historial financiero reducido. El pipeline debe consolidar indicadores financieros mensuales y permitir la exportación del reporte en formato PDF descargable.

Marco Teórico Referencial

Dominio del problema

El trabajo independiente, definido como la prestación de servicios profesionales sin relación de dependencia laboral permanente, se caracteriza por la irregularidad de ingresos, la autogestión administrativa y la responsabilidad directa sobre obligaciones fiscales (Martínez, 2019). El flujo de caja del freelancer difiere del asalariado en que carece de periodicidad fija: los

cobros dependen de la finalización de entregables, los plazos de pago del cliente y las condiciones contractuales pactadas por proyecto (López y Fernández, 2020).

En el contexto argentino, el régimen de Monotributo unifica en una cuota mensual las obligaciones de Impuesto al Valor Agregado, Impuesto a las Ganancias y aportes al sistema previsional, pero establece límites anuales de facturación por categoría que, al ser superados, fuerzan la recategorización obligatoria o la exclusión del régimen (AFIP, 2024). Ramírez (2021) documenta que la ausencia de un sistema formal de registro financiero en este segmento produce tres consecuencias medibles: subestimación de ingresos reales, incumplimiento de obligaciones tributarias por desconocimiento de montos adeudados e imposibilidad de calcular la rentabilidad efectiva de cada servicio prestado.

TICs

Python, en su versión 3.11, constituye la base del backend por su ecosistema maduro en ciencia de datos y aprendizaje automático, y por la disponibilidad de bibliotecas especializadas para procesamiento financiero. FastAPI, framework web basado en el estándar ASGI, incorpora validación de tipos en tiempo de ejecución mediante Pydantic y genera documentación interactiva conforme a la especificación OpenAPI 3.0 sin configuración adicional (Ramírez, 2021). React, en su versión 19, fue seleccionado para la capa de presentación por su modelo de componentes reutilizables y su capacidad para construir interfaces reactivas con actualización eficiente del DOM virtual.

PostgreSQL, en su versión 15, garantiza integridad referencial mediante restricciones de clave foránea, atomicidad transaccional según las propiedades ACID y soporte nativo para el tipo Numeric de precisión arbitraria, requisito para operaciones financieras donde el redondeo en aritmética de punto flotante resulta inaceptable (Codd, 1970; Momjian, 2022). Prophet, modelo

aditivo desarrollado por Meta, descompone series temporales en componentes de tendencia, estacionalidad anual, estacionalidad semanal y ruido, lo que lo hace particularmente adecuado para proyectar ingresos con datos irregulares y pocos puntos históricos (Taylor y Letham, 2018).

No obstante, el modelo requiere un volumen mínimo de observaciones para producir proyecciones estadísticamente válidas. Para los casos en que el usuario dispone de un historial financiero reducido, el sistema implementa una estrategia de arranque en frío basada en media móvil sobre los registros disponibles, con un intervalo de confianza derivado de la desviación estándar muestral. Esta aproximación, si bien menos sofisticada que el modelo completo, permite ofrecer al usuario una proyección referencial desde sus primeros registros y se reemplaza automáticamente por Prophet una vez superado el umbral mínimo de datos requerido para el ajuste del modelo.

Scikit-learn, en su versión 1.4, provee implementaciones probadas de Naive Bayes Multinomial y Support Vector Machines lineales, junto con vectorizadores TF-IDF que transforman texto en representaciones numéricas dispersas adecuadas para clasificación supervisada (Pedregosa et al., 2011).

Las prácticas de operaciones de aprendizaje automático establecen un ciclo de vida estructurado para los modelos: preparación de datos, entrenamiento, evaluación, despliegue, monitoreo y reentrenamiento (Sculley et al., 2015).

En el sistema propuesto, este ciclo se ejecuta íntegramente sobre la infraestructura local. La estrategia de serialización adopta la biblioteca joblib, recomendada por Scikit-learn por su eficiencia en la persistencia de arreglos NumPy de gran dimensionalidad típicos de los modelos entrenados con TF-IDF. El binario resultante se codifica en base64 y se almacena como texto en la columna `modelo_serializado` de la tabla `modelos_clasificador` de PostgreSQL, lo que evita migrar

el esquema a tipos LargeBinary y permite versionar múltiples modelos por usuario sin alterar la estructura relacional.

La integración con FastAPI resuelve la potencial contención de concurrencia mediante el mecanismo BackgroundTasks del framework: el reentrenamiento del modelo, activado ante correcciones del usuario o ante la acumulación de nuevos ejemplos por encima de un umbral, se ejecuta en una tarea de fondo con su propia sesión de base de datos, lo que garantiza respuesta HTTP inmediata al cliente y aislamiento de fallos del entrenamiento respecto del flujo principal de la API. Las métricas de precisión se almacenan junto con cada versión del modelo, habilitando trazabilidad histórica y comparación entre iteraciones.

El módulo de importación masiva acepta archivos en formato CSV y Excel mediante un lector unificado que selecciona el motor de Pandas según la extensión del archivo y devuelve una estructura tabular uniforme para su procesamiento posterior. La detección de la correspondencia entre las columnas del archivo y los campos del modelo de datos se resuelve con búsquedas locales: el sistema normaliza los encabezados aplicando minúsculas, eliminación de tildes y filtrado de caracteres no alfanuméricos, y los compara contra un diccionario de sinónimos relevado sobre los formatos de exportación de las principales entidades bancarias argentinas.

Esta aproximación evita transmitir el contenido del archivo a servicios externos para la inferencia de columnas y mantiene la coherencia con la política de soberanía de datos del sistema. El componente valida previamente la extensión del archivo, su tamaño máximo de diez megabytes y la presencia de contenido, delegando al usuario la confirmación de los registros detectados antes de su persistencia definitiva en la base de datos. La importación final se ejecuta bajo transacción indivisible con captura de excepciones y reversión explícita, garantizando que ningún registro parcial quede persistido en caso de error durante el procesamiento.

La protección de datos identificables en sistemas que combinan procesamiento local con servicios externos de inteligencia artificial puede abordarse mediante anonimización previa por filtros o mediante minimización del contenido transmitido. El sistema implementa la segunda estrategia, considerada más simple de auditar y más defendible que la primera: ningún texto libre proveniente del usuario, ni descripciones individuales de movimientos, ni datos identificables, se transmiten a servicios externos.

Los dos endpoints que consumen modelos de lenguaje provistos por terceros, dedicados a la generación del resumen financiero mensual y a la generación de recomendaciones personalizadas, construyen sus solicitudes exclusivamente con información numérica agregada: totales por categoría sobre una lista cerrada predefinida, conteos de movimientos, montos consolidados de facturas pendientes y porcentajes de variación entre períodos. Esta práctica se alinea con los principios de minimización de datos establecidos en la normativa argentina de protección de datos personales (Ley 25.326) y con las recomendaciones de la literatura técnica sobre privacidad por diseño en sistemas distribuidos.

Competencia

La Tabla 1 presenta una comparación entre tres soluciones de gestión contable y financiera vigentes en el mercado argentino, evaluadas según parámetros relevantes para el dominio del monotributista independiente.

Tabla 1

Comparación de soluciones competidoras en el mercado argentino

Parámetro	Contabilium	Xubio	Colppy
Orientación principal	PyMEs y contadores	PyMEs y estudios contables	PyMEs y contadores
Soporte específico	Parcial	Parcial	Parcial

Parámetro	Contabilium	Xubio	Colppy
Monotributo			
Clasificación automática de gastos por IA	No	No	No
Predicción de ingresos con ML	No	No	No
Auditoría automatizada (duplicados, anomalías)	No	No	No
Importación CSV y Excel bancarios	Sí	Sí	Sí
API pública documentada	Sí	Limitada	Sí
Modelo de costos	Suscripción mensual	Suscripción mensual	Suscripción mensual
Soberanía de datos del usuario	No declarada	No declarada	No declarada

Nota. Elaboración propia con base en la documentación pública de cada plataforma.

Diseño Metodológico

El ciclo de vida del desarrollo se gestiona mediante el marco de trabajo Scrum, seleccionado por su capacidad para acomodar requerimientos cambiantes y por su énfasis en entregas incrementales verificables. La elección responde a la naturaleza exploratoria del componente de aprendizaje automático, donde la métrica de precisión del clasificador sólo puede validarse contra datos reales y exige iteraciones sucesivas de ajuste de hiperparámetros, ampliación del dataset de entrenamiento y refinamiento del umbral de confianza. Cada sprint contempla planificación al inicio, revisión del incremento al cierre y retrospectiva para ajustes metodológicos.

Para acomodar la naturaleza experimental del componente predictivo y del clasificador de procesamiento de lenguaje natural, el cronograma incorpora sprints específicos de tipo Spike, técnica originada en programación extrema y adoptada por Scrum para tareas de investigación acotadas en tiempo.

Los Spikes del proyecto se concentran en la limpieza y curado del conjunto de datos de entrenamiento, la evaluación comparativa entre Naive Bayes Multinomial y Support Vector

Machines lineales sobre el dataset propio, el ajuste fino del umbral de confianza del clasificador y el análisis del comportamiento de Prophet bajo distintos volúmenes históricos. Estos sprints se ejecutan de forma aislada, sin integración directa al sistema principal, y entregan como artefacto un informe técnico con la decisión de diseño resultante. Una vez validadas las decisiones, los componentes se integran al sistema dentro de los sprints de desarrollo regular.

Las herramientas de desarrollo del backend comprenden Python 3.11 como lenguaje principal, FastAPI como framework web, SQLAlchemy 2.0 como mapeador objeto-relacional, Alembic para la gestión de migraciones de esquema y Pydantic para la validación de modelos de datos. El frontend se construye con React, React Router para la navegación entre vistas, Axios como cliente HTTP y Chart.js para la visualización de series temporales y proyecciones.

El componente de aprendizaje automático utiliza Scikit-learn 1.4 para los algoritmos de clasificación, Prophet 1.1 para predicción de series temporales, pandas 2.2 para manipulación de datos y joblib 1.4 para la serialización de modelos entrenados. La generación del reporte mensual en PDF se resuelve con ReportLab, biblioteca elegida por su independencia de dependencias nativas pesadas y por su modelo de composición programática.

La infraestructura se orquesta mediante Docker Compose con tres contenedores independientes para la API, la base de datos y el frontend, configuración que asegura reproducibilidad del entorno entre máquinas de desarrollo y producción.

Las técnicas de recolección de datos incluyen observación documental sobre la normativa vigente publicada por la Administración Federal de Ingresos Públicos relativa al régimen de Monotributo, el análisis comparativo de las soluciones competidoras presentadas en el marco teórico, el relevamiento de los formatos de exportación CSV y Excel de las principales entidades bancarias argentinas para la construcción del diccionario de sinónimos del importador, y la

revisión de la documentación oficial de las bibliotecas técnicas seleccionadas. La planificación temporal del proyecto se organiza en cinco fases consecutivas distribuidas en doce sprints de dos semanas cada uno, cuya distribución se presenta en la Tabla 2.

Tabla 2

Distribución de fases y sprints del proyecto

Fase	Descripción	Sprints	Entregables principales
Fase 1: Relevamiento y diseño	Análisis del dominio, modelado de datos y arquitectura del sistema.	Sprints 1-2	Diagrama de entidad-relación, modelo conceptual, especificación de endpoints.
Fase 2: Desarrollo del núcleo transaccional	Implementación del módulo CRUD financiero y autenticación.	Sprints 3-5	Endpoints de ingresos, gastos, facturas y autenticación con JWT.
Fase 3: Desarrollo del componente de aprendizaje automático (incluye Spikes)	Construcción del clasificador NLP y del motor de predicción, con sprints de investigación aislados para curado de dataset y ajuste de hiperparámetros.	Sprints 6-8	Modelo de clasificación entrenado, pipeline de reentrenamiento, predicciones Prophet, informes de Spikes técnicos.
Fase 4: Módulo fiscal y auditoría	Integración del control de Monotributo y los detectores de auditoría.	Sprints 9-10	Semáforo de recategorización, detectores de duplicados y anomalías.
Fase 5: Pruebas, reportería y documentación final	Suite de tests, generación del reporte PDF mensual y documentación académica.	Sprints 11-12	Cobertura de tests, exportación PDF, documento de tesis.

Nota. La duración de cada sprint es de dos semanas. El detalle gráfico del cronograma se presenta en el Anexo correspondiente.

Relevamiento

Relevamiento estructural

El presente proyecto se desarrolla bajo la modalidad de organización modelada, dado que el segmento objetivo está compuesto por trabajadores independientes que operan sin estructura corporativa formal, infraestructura de servidores propia ni red interna privada.

El usuario tipo del sistema desempeña su actividad profesional desde un entorno de hardware descentralizado y heterogéneo. Su equipamiento habitual consiste en una computadora

personal, ya sea de escritorio o portátil, con sistema operativo Windows, macOS o Linux, utilizada para tareas administrativas extendidas como facturación, conciliación y revisión de reportes. Complementariamente emplea un teléfono inteligente con sistema operativo Android o iOS para el registro de transacciones en el momento en que ocurren y para la consulta de indicadores financieros.

La conectividad depende de redes domésticas residenciales, redes móviles de cuarta o quinta generación y, ocasionalmente, redes Wi-Fi compartidas en espacios de coworking. El almacenamiento de información previo a la adopción del sistema se distribuye entre el disco local de cada dispositivo, servicios de nube genéricos y el correo electrónico utilizado como repositorio informal de comprobantes.

Relevamiento funcional

Los roles intervinientes en el proceso financiero del monotributista son tres. El Freelancer constituye el actor principal, responsable del registro, control y declaración de movimientos. El Cliente representa la contraparte contractual que solicita servicios, recibe la factura y efectúa el pago correspondiente. La Administración Federal de Ingresos Públicos (ARCA) actúa como autoridad fiscal que valida la facturación electrónica, define las categorías del régimen y recibe la cuota mensual de Monotributo.

El proceso manual actual de Control de Facturación, ejecutado sin la asistencia del sistema propuesto, se desarrolla del siguiente modo: el Freelancer presta el servicio acordado al Cliente y, una vez finalizado el entregable, accede al portal web de la Administración Federal de Ingresos Públicos para emitir la factura electrónica correspondiente.

La autoridad fiscal valida la emisión y entrega un comprobante con el Código de Autorización Electrónico. El Freelancer descarga el archivo en formato PDF y lo remite al Cliente

por correo electrónico. El Cliente recibe la factura, ejecuta el pago según los plazos pactados y notifica la transferencia.

El Freelancer registra manualmente el cobro en una hoja de cálculo, junto con la fecha, el monto y el cliente asociado. Mensualmente, el profesional debe consolidar manualmente la facturación acumulada del año y compararla con el límite de su categoría de Monotributo para determinar si corresponde recategorización; en paralelo, debe verificar que el pago de la cuota mensual del régimen se haya efectuado dentro del plazo establecido por el organismo recaudador.

Las inconsistencias de este flujo, como facturas vencidas sin cobrar, gastos cargados por duplicado o anomalías en patrones de gasto, no son detectadas por ningún mecanismo automatizado y dependen del cruce manual entre comprobantes, extractos bancarios y planillas.

Relevamiento de documentación

La documentación involucrada en el proceso financiero del monotributista comprende la Factura Electrónica Tipo C, comprobante emitido por sujetos inscriptos en el régimen simplificado para operaciones con consumidores finales y otros monotributistas, y la Constancia de Inscripción al Monotributo, documento oficial que acredita la categoría vigente del contribuyente. A estos documentos se suman los comprobantes de pago de la cuota mensual, los extractos bancarios mensuales y los archivos CSV y Excel exportados desde aplicaciones de homebanking. El detalle de estructura, formato y frecuencia de cada documento se desarrolla en el Anexo correspondiente.

Procesos de Negocios

El proceso manual de Control de Facturación, descrito en el relevamiento funcional, involucra la interacción secuencial entre tres actores principales: el Freelancer como ejecutor del

registro y control, el Cliente como contraparte contractual y la Administración Federal de Ingresos Públicos como autoridad fiscal.

La representación gráfica del flujo se desarrolla mediante un diagrama BPMN (Business Process Model and Notation) con tres carriles diferenciados, uno por cada rol interviniente, donde se muestran las actividades secuenciales, los puntos de decisión y los artefactos documentales intercambiados entre los actores. El diagrama permite visualizar la dependencia temporal entre la emisión de la factura electrónica, el pago del cliente, el registro manual del cobro y la consolidación periódica con fines fiscales.

[Insertar Figura: Diagrama BPMN del proceso manual de Control de Facturación]

Diagnóstico y propuesta

A partir del relevamiento funcional desarrollado en la primera entrega, se identificaron los problemas y causas asociadas al proceso manual de Control de Facturación que ejecuta el monotributista en su gestión cotidiana. La Tabla 3 sintetiza el diagnóstico, distinguiendo los problemas observables de las causas estructurales que los producen, conforme al esquema metodológico solicitado para este apartado.

Tabla 3

Diagnóstico del proceso Control de Facturación

Proceso	Problema	Causa
Control de Facturación	Pérdida de visibilidad sobre el acumulado anual de facturación.	Los movimientos se registran en planillas dispersas que el freelancer consolida manualmente, lo que produce sumas parciales tardías respecto del plazo de recategorización fiscal.
Control de Facturación	Riesgo no anticipado de recategorización en el régimen de Monotributo.	El monotributista carece de herramientas que proyecten la facturación futura a partir de su historial propio, por lo que descubre el riesgo de exceder el límite de su categoría únicamente al final del período fiscal.
Control de Facturación	Inconsistencias no detectadas en los registros financieros.	La revisión cruzada entre comprobantes, extractos bancarios y planillas se realiza únicamente al cierre fiscal anual, en lugar de aplicarse de forma continua sobre cada movimiento ingresado.
Control de Facturación	Carga manual repetitiva al consolidar movimientos bancarios.	Las entidades bancarias entregan los movimientos exclusivamente en archivos exportados con formatos heterogéneos entre proveedores, lo que obliga al freelancer a transcribir manualmente cada movimiento a su planilla personal.
Control de Facturación	Clasificación inconsistente de los gastos profesionales.	La categorización de cada movimiento se asigna según el criterio subjetivo del freelancer en el momento del registro, sin patrones predefinidos ni aprendizaje sobre las decisiones previas del propio usuario.
Control de Facturación	Ausencia de información consolidada para la toma de decisiones financieras del propio freelancer.	Los reportes que el monotributista necesita para evaluar su rentabilidad mensual, las facturas pendientes de cobro y la evolución de su flujo de caja se construyen en forma manual a partir de fuentes desagregadas, lo que produce reportes intermitentes y de

Proceso	Problema	Causa
		calidad variable.

Nota. Elaboración propia a partir del relevamiento funcional de la primera entrega.

Propuesta

Se propone el desarrollo de un sistema web denominado FreelanceControl que asuma de manera integrada las tareas que hoy resuelve el monotributista en forma manual, fragmentada y reactiva. El sistema centraliza el registro de ingresos, gastos y facturas en una base de datos relacional con aislamiento por usuario autenticado, lo que elimina la dispersión documental entre planillas, comprobantes y aplicaciones genéricas. Sobre los movimientos registrados se aplican tres componentes automatizados que abordan directamente las causas estructurales identificadas en el diagnóstico.

El primer componente es un clasificador de gastos basado en procesamiento de lenguaje natural, entrenado localmente con algoritmos Naive Bayes y Support Vector Machines sobre vectorización TF-IDF, que asigna automáticamente una categoría a cada gasto a partir de su descripción y aprende de las correcciones que el usuario aplica sobre clasificaciones previas. El segundo componente es un motor de predicción de ingresos basado en el modelo Prophet, que proyecta el flujo de caja a seis meses con intervalos de confianza y, en función de esa proyección, calcula el porcentaje proyectado de utilización del límite anual de la categoría de Monotributo del usuario, generando alertas tempranas cuando ese porcentaje supera umbrales preestablecidos. El tercer componente es un módulo de auditoría automatizada que detecta gastos duplicados por ventana temporal, anomalías estadísticas mediante puntaje z, facturas vencidas sin cobrar y la falta de registro del pago mensual de la cuota de Monotributo, transformando una revisión que hoy se realiza de manera anual en un control continuo y sistemático.

Adicionalmente, el sistema dispone de un módulo de importación masiva multiformato que acepta archivos CSV y Excel exportados por las principales entidades bancarias argentinas, con detección heurística local de la estructura del archivo y persistencia transaccional atómica con reversión ante errores. Finalmente, un módulo de reportería genera un documento PDF mensual descargable que consolida los indicadores financieros del período. La arquitectura completa se construye sobre FastAPI, PostgreSQL y React, orquestada mediante Docker Compose, con un núcleo de procesamiento local que garantiza la soberanía sobre la información financiera del usuario y restringe el uso de modelos de lenguaje externos a la generación de resúmenes y recomendaciones a partir de información numérica agregada.

Objetivo, límite y alcances del prototipo

Objetivo del prototipo

Desarrollar un prototipo funcional del sistema FreelanceControl que permita al monotributista argentino registrar ingresos, gastos y facturas con clasificación automatizada por procesamiento de lenguaje natural, proyectar el flujo de caja a seis meses mediante un motor estadístico de series temporales, monitorear el estado fiscal del régimen de Monotributo a partir de la facturación acumulada y proyectada, y auditar los registros financieros mediante detectores automatizados de inconsistencias.

Límites

El alcance del prototipo se delimita desde el registro autenticado del primer movimiento financiero por parte del usuario hasta la generación del reporte PDF mensual consolidado del período fiscal en curso. Quedan fuera del límite del prototipo la integración con sistemas externos

de facturación electrónica del organismo recaudador, la conciliación bancaria automatizada con cuentas reales del usuario y la gestión de clientes como entidad independiente desacoplada de las facturas emitidas.

Alcances

Los procesos cubiertos por el prototipo dentro del límite definido son los siguientes:

- Registro y autenticación de usuarios con aislamiento de datos por sesión.
- Gestión transaccional de ingresos, gastos y facturas con máquina de estados.
- Clasificación automatizada de gastos mediante procesamiento de lenguaje natural local.
- Reentrenamiento del clasificador a partir de correcciones manuales del usuario.
- Importación masiva multiformato de movimientos desde archivos CSV y Excel.
- Auditoría automatizada de duplicados, anomalías estadísticas y facturas vencidas.
- Monitoreo del estado fiscal de la categoría de Monotributo con semáforo de riesgo.
- Proyección de ingresos a seis meses con intervalos de confianza.
- Generación de resúmenes financieros y recomendaciones en lenguaje natural.
- Exportación del reporte mensual consolidado en formato PDF descargable.

Descripción del sistema

Conforme al diseño metodológico declarado en la primera entrega, la descripción del prototipo se desarrolla mediante artefactos del marco de trabajo Scrum. Se presentan a continuación el Product Backlog, las Historias de Usuario priorizadas con sus criterios de aceptación y el Sprint Backlog correspondiente al primer sprint del desarrollo.

Product Backlog

El Product Backlog consolida los elementos de trabajo identificados para el prototipo, ordenados de mayor a menor prioridad y caracterizados por una estimación relativa de esfuerzo expresada en puntos de historia. La Tabla 4 presenta el backlog completo.

Tabla 4

Product Backlog del prototipo FreelanceControl

ID	Descripción	Prioridad	Esfuerzo (puntos)
PB-01	Registro y autenticación de usuarios con JSON Web Token.	Alta	5
PB-02	Gestión CRUD de ingresos con filtros y paginación.	Alta	5
PB-03	Gestión CRUD de gastos con clasificación automática.	Alta	8
PB-04	Gestión CRUD de facturas con máquina de estados.	Alta	8
PB-05	Clasificador NLP local con Naive Bayes y SVM.	Alta	13
PB-06	Reentrenamiento automático del clasificador ante correcciones del usuario.	Alta	8
PB-07	Importación masiva CSV y Excel con detección heurística.	Media	13
PB-08	Auditoría automatizada: duplicados, anomalías, vencidas, monotributo impago.	Alta	13
PB-09	Proyección de ingresos con Prophet y cold start.	Alta	13
PB-10	Monitoreo del estado fiscal de Monotributo con semáforo de riesgo.	Alta	8
PB-11	Resumen financiero mensual en lenguaje natural.	Media	5
PB-12	Recomendaciones personalizadas sobre datos agregados.	Media	5
PB-13	Generación de reporte PDF mensual descargable.	Media	8
PB-14	Interfaz frontend con 13 pantallas y diseño de tema oscuro.	Alta	21
PB-15	Migración versionada del esquema de base de datos con Alembic.	Media	3
PB-16	Suite automatizada de pruebas con pytest e infraestructura de testing.	Media	8
PB-17	Contenedorización con Docker Compose para reproducibilidad de entornos.	Media	5

Nota. Estimación de esfuerzo según la sucesión modificada de Fibonacci (1, 2, 3, 5, 8, 13, 21) habitualmente utilizada en planning poker.

Historias de usuario

A continuación se presentan las historias de usuario correspondientes a los elementos del Product Backlog, formuladas según el esquema estándar de Mike Cohn (rol, funcionalidad, beneficio) con sus respectivos criterios de aceptación.

HU-01: Registro de usuario

Como monotributista que aún no usa el sistema quiero registrarme con mi nombre, correo electrónico y contraseña para acceder a una cuenta personal aislada del resto de los usuarios.

Criterios de aceptación:

- El sistema valida que el correo electrónico tenga formato válido.
- El sistema rechaza el registro si el correo ya está en uso, indicando el motivo del rechazo.
- El sistema almacena la contraseña aplicando hash con bcrypt, nunca en texto plano.
- El sistema retorna un mensaje de confirmación una vez completado el registro.

HU-02: Inicio de sesión

Como monotributista registrado quiero iniciar sesión con mi correo electrónico y contraseña para acceder a mis datos financieros desde cualquier dispositivo.

Criterios de aceptación:

- El sistema valida las credenciales contra la base de datos.
- El sistema devuelve el mismo mensaje de error si el correo no existe o si la contraseña es incorrecta, para evitar enumeración de usuarios.
- El sistema emite un token JSON Web Token con vigencia de siete días al validar las credenciales.

- El sistema redirige al usuario a su panel principal una vez autenticado.

HU-03: Registro de ingresos

*Como **monotributista autenticado** quiero **registrar un ingreso con su descripción, monto, categoría y fecha para consolidar mis cobros profesionales en una única vista.***

Criterios de aceptación:

- El sistema obtiene el identificador del usuario exclusivamente del token de autenticación, nunca del cuerpo del pedido.
- El sistema valida que el monto sea estrictamente positivo.
- El sistema confirma la creación del registro mediante un código HTTP 201.
- El sistema lista los ingresos del usuario ordenados por fecha descendente.

HU-04: Registro de gastos con clasificación automática

*Como **monotributista autenticado** quiero **registrar un gasto ingresando solo su descripción y monto para que el sistema sugiera automáticamente la categoría correspondiente.***

Criterios de aceptación:

- El sistema invoca al clasificador local de procesamiento de lenguaje natural para inferir la categoría a partir de la descripción.
- El sistema indica la categoría sugerida junto con un valor de confianza asociado.
- Si el valor de confianza es inferior al umbral de 0,30, el sistema asigna la categoría Otros y marca el registro como sujeto a revisión.
- El usuario puede confirmar o modificar la categoría sugerida antes de persistir el gasto.

HU-05: Corrección manual de categorías

*Como **monotributista autenticado** quiero **modificar la categoría asignada por el clasificador a un gasto previamente registrado para que el sistema aprenda de mis correcciones y mejore en clasificaciones futuras.***

Criterios de aceptación:

- El sistema persiste la modificación en la base de datos.
- El sistema dispara un proceso de reentrenamiento del modelo cuando detecta un cambio de categoría.
- El reentrenamiento se ejecuta en una tarea de fondo sin bloquear la respuesta al usuario.
- El sistema almacena la nueva versión del modelo entrenado en la base de datos.

HU-06: Gestión del ciclo de vida de facturas

*Como **monotributista autenticado** quiero **registrar facturas emitidas a clientes con sus fechas de emisión y vencimiento para controlar el estado de cobro de cada factura emitida.***

Criterios de aceptación:

- El sistema asigna el estado inicial Pendiente al crear una nueva factura.
- El sistema permite transicionar el estado a Pagada al registrar la fecha de pago efectivo.
- El módulo de auditoría identifica las facturas pendientes cuya fecha de vencimiento fue superada y genera una alerta de discrepancia de facturación.
- El sistema impide editar o eliminar facturas en estado Pagada, retornando un código HTTP 409.

HU-07: Importación masiva desde el banco

*Como **monotributista autenticado** quiero **importar mis movimientos bancarios desde un archivo CSV o Excel exportado por mi banco para evitar la carga manual de cada movimiento individual.***

Criterios de aceptación:

- El sistema valida que el archivo tenga extensión CSV o XLSX antes de procesarlo.
- El sistema rechaza archivos superiores a diez megabytes mediante código HTTP 413.
- El sistema infiere mediante heurísticas locales la correspondencia entre las columnas del archivo y los campos del modelo de datos.
- El sistema presenta una vista previa con los primeros veinte registros antes de la confirmación final.
- La importación se ejecuta como transacción atómica: si falla algún registro, se revierten todos los anteriores.

HU-08: Auditoría automatizada de los registros

*Como **monotributista autenticado** quiero **ejecutar bajo demanda una auditoría sobre mis registros financieros para detectar gastos duplicados, anomalías, facturas vencidas y pagos pendientes de Monotributo sin intervención manual.***

Criterios de aceptación:

- El sistema detecta gastos con mismo monto y categoría dentro de una ventana de tres días.

- El sistema identifica gastos cuyo monto presente un puntaje z superior a 2,0 respecto de la media de su categoría.
- El sistema identifica facturas en estado Pendiente cuya fecha de vencimiento haya sido superada.
- El sistema verifica la ausencia de un gasto en la categoría Monotributo dentro del mes en curso y genera una alerta cuando el pago de la cuota no fue registrado.
- El sistema borra las alertas no resueltas previas antes de regenerarlas, evitando duplicación.
- Las alertas marcadas como resueltas se conservan como historial auditable.

HU-09: Proyección de ingresos a seis meses

*Como **monotributista autenticado** quiero **obtener una proyección de mis ingresos para los próximos seis meses para anticipar mi flujo de caja y planificar decisiones financieras.***

Criterios de aceptación:

- El sistema utiliza el modelo Prophet cuando el usuario dispone de diez o más ingresos históricos.
- El sistema aplica una estrategia de arranque en frío basada en media móvil cuando el historial es inferior a diez registros.
- Cada proyección mensual se acompaña de un intervalo de confianza con límite inferior y superior.
- El sistema reemplaza las proyecciones anteriores al regenerar, evitando acumulación de versiones.

HU-10: Monitoreo del estado fiscal

Como monotributista autenticado quiero consultar el porcentaje de utilización del límite anual de mi categoría de Monotributo para anticipar el riesgo de recategorización antes del cierre fiscal.

Criterios de aceptación:

- El sistema calcula el acumulado real combinando todos los ingresos del año en curso.
- El sistema suma la proyección Prophet hasta el cierre del año fiscal para estimar el total anual.
- El sistema asigna un semáforo verde si el porcentaje proyectado es inferior al 70%, amarillo entre 70% y 90% y rojo si supera el 90%.
- El sistema sugiere la categoría siguiente si la proyección supera el límite vigente.

HU-11: Resumen financiero en lenguaje natural

Como monotributista autenticado quiero consultar un resumen narrativo de mi situación financiera del mes para comprender mi estado financiero sin necesidad de interpretar tablas o gráficos.

Criterios de aceptación:

- El sistema construye el resumen exclusivamente a partir de información numérica agregada del usuario.
- El sistema no transmite descripciones individuales, nombres de clientes ni datos identificables a servicios externos.
- El resumen tiene una extensión máxima de 150 palabras.

- Ante fallos del servicio externo, el sistema utiliza un fallback local con plantillas predefinidas.

HU-12: Recomendaciones personalizadas

Como monotributista autenticado quiero recibir recomendaciones financieras accionables basadas en mi estado actual para tomar decisiones informadas sobre mi gestión profesional.

Criterios de aceptación:

- El sistema genera entre tres y cinco recomendaciones por sesión.
- Las recomendaciones se construyen a partir de los datos agregados del usuario, sin texto libre identificable.
- Las recomendaciones consideran simultáneamente alertas activas, tendencias de ingreso y proyecciones.

HU-13: Descarga del reporte mensual en PDF

Como monotributista autenticado quiero descargar un reporte PDF con el resumen financiero del mes cerrado para archivar o compartir el reporte con mi contador.

Criterios de aceptación:

- El sistema genera el archivo con encabezado, resumen ejecutivo, estado fiscal, distribución de gastos, facturación y auditoría.
- El sistema utiliza el formato argentino de pesos con punto como separador de miles y coma como separador decimal.

- El sistema incluye un disclaimer indicando que el documento no reemplaza el asesoramiento de un contador matriculado.
- El archivo se entrega con cabecera Content-Disposition para forzar la descarga en el navegador.

Sprint Backlog del primer sprint

El primer sprint del proyecto se planifica con una duración de dos semanas y un objetivo concreto: completar el núcleo transaccional del sistema, sobre el cual se construirán posteriormente los módulos de aprendizaje automático, auditoría y reportería. La Tabla 5 presenta las historias seleccionadas con su descomposición en tareas técnicas y la estimación temporal asociada.

Tabla 5

Sprint Backlog del primer sprint

Historia	Tarea técnica	Responsable	Estimación (horas)
HU-01 / HU-02	Diseñar el esquema de la tabla usuarios y la migración Alembic inicial.	Desarrollador backend	3
HU-01	Implementar endpoint POST /auth/register con hash bcrypt y validación de email único.	Desarrollador backend	4
HU-02	Implementar endpoint POST /auth/login con OAuth2PasswordRequestForm y emisión de JWT.	Desarrollador backend	4
HU-02	Configurar dependencia get_current_user con validación de token y resguardo de endpoints.	Desarrollador backend	3
HU-03	Diseñar esquemas Pydantic IngresoCreate, IngresoResponse y migración de la tabla ingresos.	Desarrollador backend	2
HU-03	Implementar endpoints CRUD del recurso ingresos con filtros, paginación y ownership.	Desarrollador backend	6
HU-04	Implementar endpoints CRUD del recurso gastos sin clasificación (preparación de la base).	Desarrollador backend	4
HU-06	Implementar endpoints CRUD de facturas con máquina de estados y bloqueo de edición sobre Pagada.	Desarrollador backend	6
Transversal	Configurar la infraestructura de testing con pytest, fixtures de SQLite in-memory y TestClient.	Desarrollador backend	4

Historia	Tarea técnica	Responsable	Estimación (horas)
Transversal	Escribir tests automatizados de autenticación, ingresos, gastos y facturas.	Desarrollador backend	6
Transversal	Configurar Docker Compose con tres contenedores (api, db, frontend) y variables de entorno.	Desarrollador backend	4
HU-01 a HU-06	Implementar las pantallas de Login, Registro, Dashboard, Ingresos, Gastos y Facturas en React.	Desarrollador frontend	18
HU-01 a HU-06	Integrar el cliente Axios con interceptor de JWT y manejo de sesión expirada.	Desarrollador frontend	4
Transversal	Revisión cruzada, refactoring y reunión de revisión del sprint.	Equipo completo	4

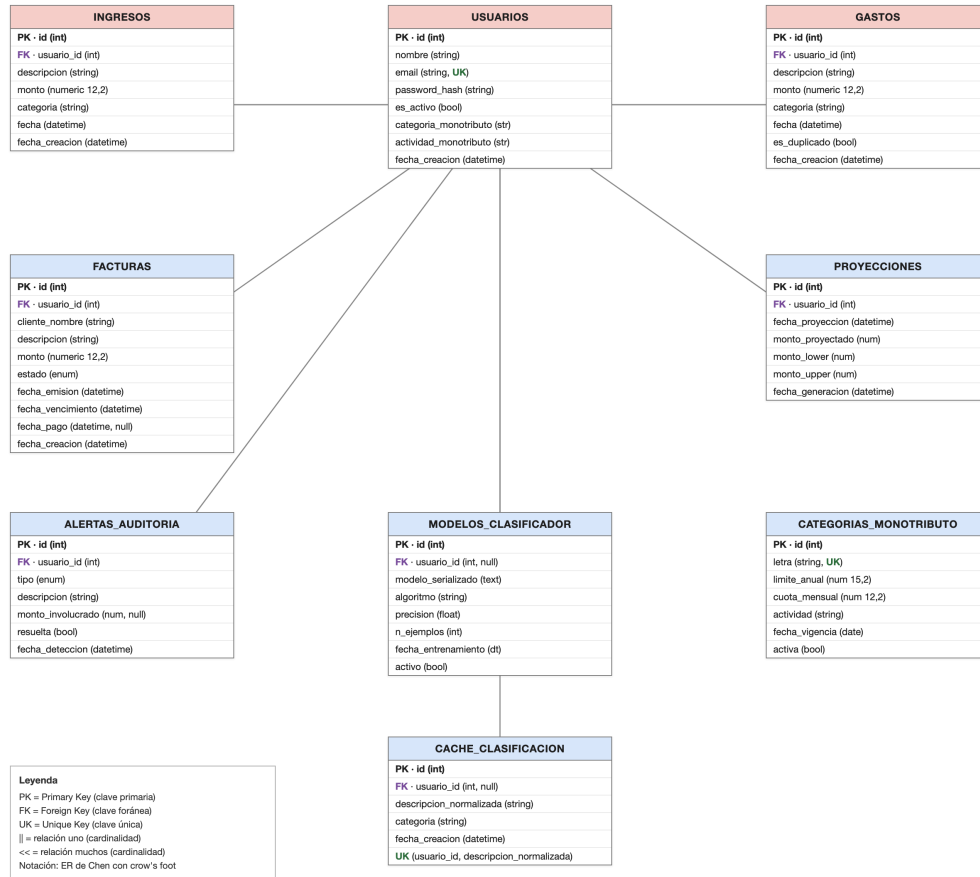
Nota. Estimación temporal expresada en horas-persona. Total del sprint: 72 horas distribuidas entre ambos roles.

Estructura de datos

El sistema utiliza una base de datos relacional PostgreSQL versión 15 con un esquema compuesto por nueve tablas, gestionado mediante el mapeador objeto-relacional SQLAlchemy 2.0 y versionado con migraciones Alembic. La Figura 1 presenta el diagrama de entidad-relación con las tablas, sus atributos principales, claves y relaciones.

Figura 1

Diagrama de entidad-relación del sistema FreelanceControl



Nota. La tabla USUARIOS es el centro del modelo: todas las tablas transaccionales (INGRESOS, GASTOS, FACTURAS, PROYECCIONES, ALERTAS_AUDITORIA, MODELOS_CLASIFICADOR y CACHE_CLASIFICACION) mantienen una relación de uno a muchos con ella. La tabla CATEGORIAS_MONOTRIBUTO actúa como catálogo fiscal versionado, y CACHE_CLASIFICACION almacena las correcciones de categoría aportadas por el usuario para el reentrenamiento del clasificador.

Diccionario de datos

El diccionario de datos describe los campos de las tablas que forman el núcleo del negocio del sistema. La Tabla 6 documenta los campos de la tabla GASTOS, central para la operación del clasificador NLP y del módulo de auditoría.

Tabla 6

Diccionario de datos de la tabla GASTOS

Campo	Tipo de dato	Descripción
id	Integer (PK)	Identificador único autoincremental del gasto.
usuario_id	Integer (FK)	Referencia al usuario propietario del gasto. Aplica restricción NOT NULL.
descripcion	String(255)	Descripción textual del gasto utilizada como entrada del clasificador NLP.
monto	Numeric(12,2)	Importe del gasto en pesos argentinos con precisión de dos decimales.
categoria	String(100)	Categoría asignada por el clasificador o por corrección manual del usuario. Una de las doce categorías predefinidas.
fecha	DateTime con zona	Fecha de ocurrencia del gasto. Utilizada en las ventanas temporales de auditoría.
es_duplicado	Boolean	Bandera establecida por el módulo de auditoría cuando detecta un gasto candidato a duplicación.
fecha_creacion	DateTime con zona	Marca temporal de creación del registro, gestionada automáticamente por la base de datos.

Nota. El tipo Numeric(12,2) garantiza precisión exacta en operaciones monetarias, evitando el redondeo de la aritmética de punto flotante.

Tabla 7

Diccionario de datos de la tabla MODELOS_CLASIFICADOR

Campo	Tipo de dato	Descripción
id	Integer (PK)	Identificador único autoincremental del modelo.
usuario_id	Integer (FK, nullable)	Referencia al usuario propietario del modelo. Si es nulo, indica modelo base global compartido.
modelo_serializado	Text	Modelo entrenado serializado con joblib y codificado en base64 para persistencia como texto.
algoritmo	String	Algoritmo utilizado para entrenar el modelo: naive_bayes o svm.
precision	Float	Precisión global del modelo medida sobre el conjunto de validación.
n_ejemplos	Integer	Cantidad de ejemplos de entrenamiento utilizados al construir el modelo.
fecha_entrenamiento	DateTime	Marca temporal de creación del modelo, utilizada para trazabilidad histórica.
activo	Boolean	Indica si el modelo es el actualmente en uso para inferencia.

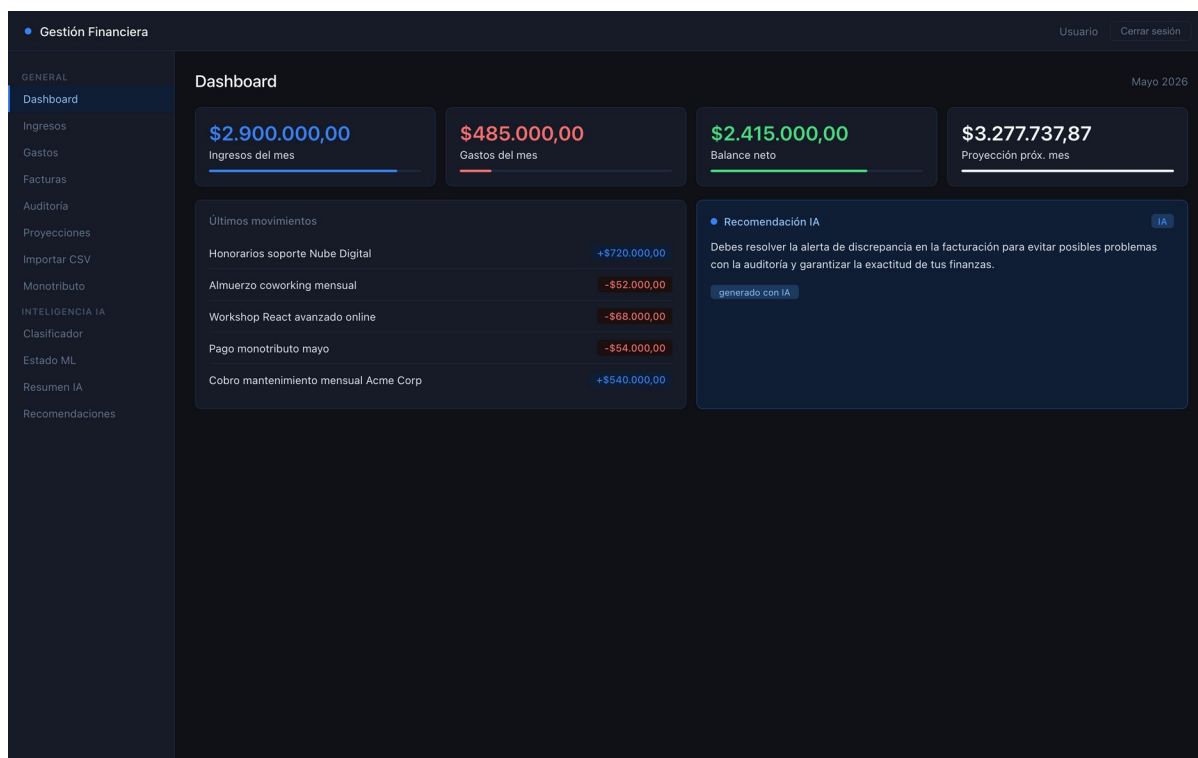
Nota. La persistencia del modelo como texto base64 evita migrar la columna al tipo LargeBinary y mantiene compatibilidad con herramientas de inspección de bases de datos relacionales.

Prototipos de interfaces de pantallas

A continuación se presentan tres capturas de pantalla de la interfaz del prototipo en su versión funcional. Las pantallas seleccionadas corresponden a los procesos núcleo del negocio: el panel principal con consolidación financiera, el registro de gastos con clasificación automatizada, y el monitoreo del estado fiscal de Monotributo. Las capturas mantienen coherencia directa con las historias de usuario HU-03, HU-04 y HU-10 descritas previamente.

Figura 2

Panel principal del sistema FreelanceControl



Nota. Captura del frontend en ejecución (React 19). Refleja la consolidación de ingresos, gastos y facturas del período en curso, coherente con la historia HU-03.

Figura 3

Pantalla de gestión de gastos con clasificación NLP

Gestión Financiera

Usuario

Cerrar sesión

GENERAL

Dashboard

Ingresos

Gastos

Facturas

Auditoría

Proyecciones

Importar CSV

Monotributo

INTELIGENCIA IA

Clasificador

Estado ML

Resumen IA

Recomendaciones

Gastos

Todas las categorías

☐ Solo duplicados

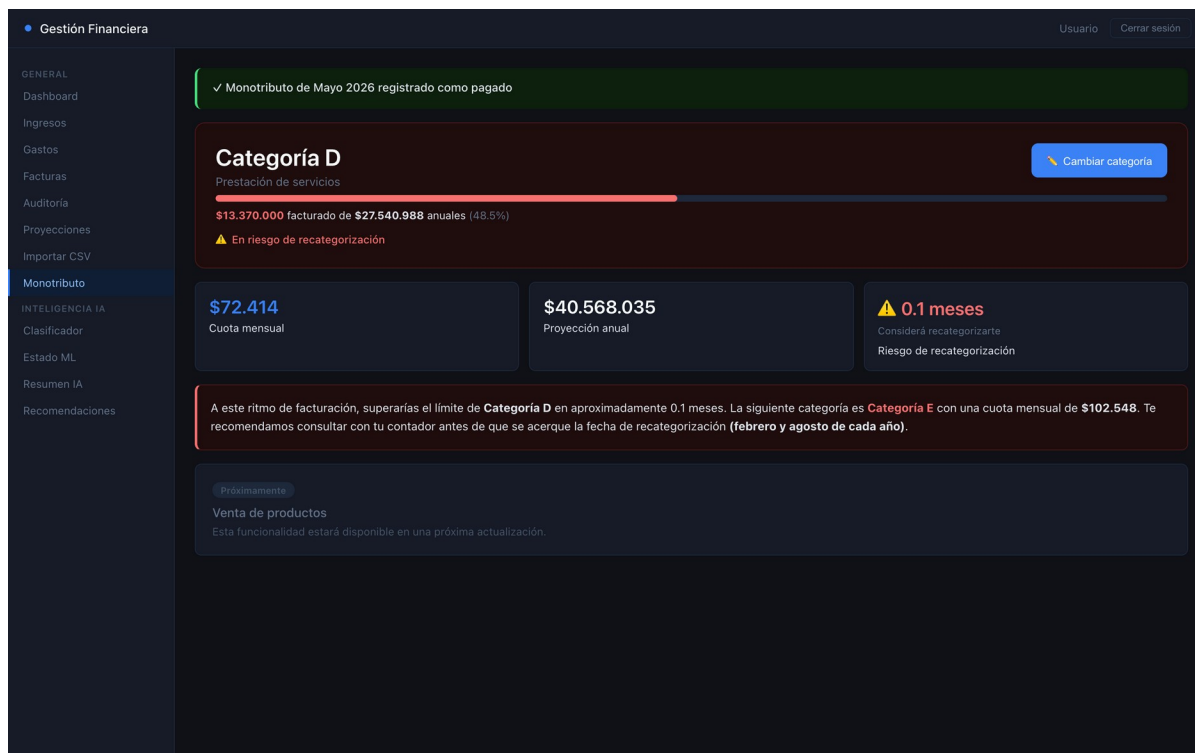
+ Nuevo gasto

DESCRIPCIÓN	CATEGORÍA	FECHA	MONTO	
Almuerzo coworking mensual	Alimentación	21 may 2026	-\$52.000,00	Eliminar
Workshop React avanzado online	Capacitación	18 may 2026	-\$68.000,00	Eliminar
Pago monotributo mayo	Monotributo	15 may 2026	-\$54.000,00	Eliminar
Honorarios contadora mayo	Servicios	12 may 2026	-\$95.000,00	Eliminar
AWS EC2 hosting mensual	Infraestructura	9 may 2026	-\$104.000,00	Eliminar
Dominio y certificado SSL anual	Infraestructura	6 may 2026	-\$72.000,00	Eliminar
Suscripción Adobe Creative Cloud	Suscripciones	3 may 2026	-\$40.000,00	Eliminar
Uber viajes a reuniones	Transporte	21 abr 2026	-\$28.000,00	Eliminar
Teclado y mouse Logitech	Hardware	18 abr 2026	-\$96.000,00	Eliminar
Pago monotributo abril	Monotributo	15 abr 2026	-\$51.000,00	Eliminar
Honorarios contadora abril	Servicios	12 abr 2026	-\$90.000,00	Eliminar
AWS EC2 hosting mensual	Infraestructura	9 abr 2026	-\$101.000,00	Eliminar
Ingresos brutos CABA declaración	Impuestos	6 abr 2026	-\$138.000,00	Eliminar
Suscripción Adobe Creative Cloud	Suscripciones	3 abr 2026	-\$39.000,00	Eliminar
Cena cliente restaurante	Alimentación	21 mar 2026	-\$46.000,00	Eliminar
Publicidad Google Ads campaña	Marketing	18 mar 2026	-\$120.000,00	Eliminar

Nota. Captura de la pantalla de gastos. Cada movimiento muestra la categoría asignada por el clasificador local, coherente con la historia HU-04.

Figura 4

Pantalla de monitoreo del estado fiscal de Monotributo



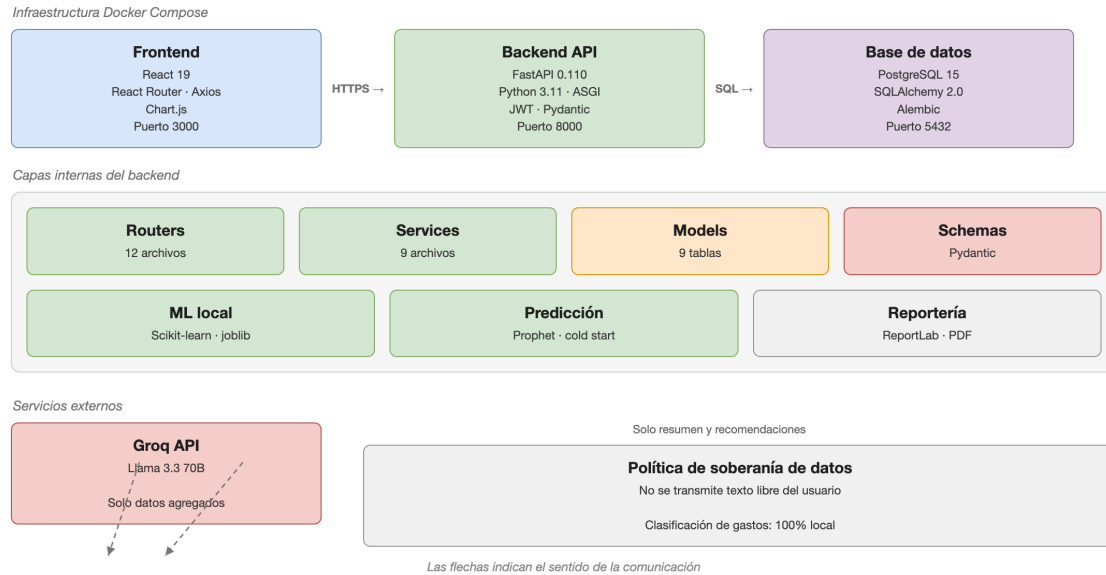
Nota. Captura de la pantalla de Monotributo. Muestra el porcentaje proyectado de utilización del límite anual y el semáforo de riesgo, coherente con la historia HU-10.

Diagrama de arquitectura

La Figura 5 presenta la arquitectura general del sistema FreelanceControl. La solución se compone de tres contenedores Docker independientes que se comunican mediante una red interna definida en Docker Compose: el contenedor frontend que aloja la aplicación React, el contenedor backend que ejecuta la API construida con FastAPI, y el contenedor de base de datos que gestiona la instancia de PostgreSQL. El backend se organiza internamente en cuatro capas (routers, services, models y schemas) que separan la responsabilidad de cada componente, e incorpora tres servicios especializados de aprendizaje automático y reportería que se ejecutan dentro del mismo proceso. La comunicación con servicios externos se limita a la API de Groq y se utiliza exclusivamente para la generación de resúmenes y recomendaciones a partir de datos agregados, manteniendo la política de soberanía sobre los datos identificables del usuario.

Figura 5

Arquitectura general del sistema FreelanceControl



Nota. Elaboración propia. Las flechas continuas indican comunicación interna entre contenedores. La flecha discontinua indica la comunicación con el servicio externo Groq, acotada según la política de soberanía de datos.

Seguridad

La seguridad de FreelanceControl se aborda desde dos planos complementarios: el control de acceso a la aplicación y la política de respaldo de la información. Ambos se describen a partir de los mecanismos efectivamente implementados en el sistema y de las decisiones de despliegue previstas para su puesta en producción.

Acceso a la aplicación. El ingreso al sistema se resuelve mediante autenticación basada en JSON Web Tokens (JWT). En el registro, el correo electrónico actúa como identificador único: el sistema verifica que no exista previamente en la base de datos y rechaza el alta en caso contrario, además de validar que su formato sea correcto. La contraseña exigida posee una longitud mínima de ocho caracteres y nunca se almacena en texto plano; se persiste como un hash generado con el algoritmo bcrypt, que incorpora un valor de sal aleatorio por cada registro, de modo que dos contraseñas idénticas producen hashes distintos y resultan inservibles ante una eventual filtración de la base.

En el inicio de sesión, el sistema compara las credenciales recibidas contra el hash almacenado. Ante un correo inexistente o una contraseña incorrecta devuelve siempre el mismo mensaje de error genérico, decisión deliberada para impedir la enumeración de usuarios válidos por parte de un atacante. Si la cuenta se encuentra desactivada, se deniega el acceso aun cuando las credenciales sean correctas. Validadas las credenciales, el sistema emite un token firmado con el algoritmo HMAC-SHA256 y una vigencia de siete días; la clave de firma se mantiene fuera del código fuente, en una variable de entorno. Cada solicitud posterior viaja con ese token en el encabezado de autorización, y el servidor deriva de él la identidad del usuario. Un principio transversal del diseño es que el identificador del usuario se obtiene siempre del token y nunca del cuerpo de la petición, lo que evita que un cliente manipule la pertenencia de los datos.

Manejo de perfiles. El dominio del sistema involucra un único tipo de usuario operativo: el monotributista autenticado, que administra exclusivamente su propia información. Al tratarse de una herramienta de uso individual, no se definieron roles diferenciados como administrador o invitado, ya que no responden a una necesidad real del problema abordado. La protección esencial en este punto es el aislamiento por usuario: todas las consultas a la base de datos se filtran por el identificador obtenido del token, de manera que ningún usuario puede consultar ni modificar registros que no le pertenezcan. De este modo, la separación entre cuentas se garantiza a nivel de cada operación y no depende de la interfaz.

Política de respaldo de información. El sistema se despliega sobre un servidor virtual privado (VPS) en la nube, orquestado con Docker Compose en tres contenedores —aplicación, base de datos y frontend—, y la totalidad de la información persiste en la base de datos PostgreSQL. El resguardo se concibe como un proceso automático y programado, sin intervención manual en su ejecución cotidiana.

El proceso se inicia mediante una tarea programada (cron) en el servidor, que ejecuta diariamente, en el horario de menor actividad, un volcado lógico de la base de datos con la utilidad `pg_dump` nativa de PostgreSQL. El software interviniente comprende, entonces, `pg_dump` para la generación de la copia, el planificador cron para su disparo y un cliente de sincronización para el envío de las copias al almacenamiento externo. El hardware involucrado es el propio VPS, donde reside la copia primaria, y el servicio de almacenamiento de objetos del proveedor de nube, donde se aloja la copia remota. El factor humano se limita a un responsable de datos que supervisa la integridad de los respaldos y realiza, de manera periódica, restauraciones de prueba para verificar que las copias sean efectivamente recuperables.

En cuanto al esquema de redundancia y almacenamiento, se conservan tres niveles de copia: una copia primaria diaria en el VPS; una copia diaria replicada en un servicio de almacenamiento de objetos en la nube, con una política de retención de treinta días que elimina automáticamente las más antiguas; y una copia semanal adicional descargada a un medio externo bajo control del responsable de datos. El código fuente, por su parte, se versiona en un repositorio Git remoto, que constituye su propio mecanismo de respaldo y trazabilidad ante cada actualización. Esta política contempla los tres pilares de la seguridad de la información: la disponibilidad, sostenida por la redundancia de copias y la infraestructura del proveedor; la integridad, verificada mediante restauraciones de prueba; y la confidencialidad, preservada tanto por el acceso restringido a las copias como por la política de soberanía de datos del sistema, que evita exponer información sensible a terceros.

Análisis de Costos

El análisis de costos contempla dos dimensiones: los costos de desarrollo, asociados al esfuerzo humano necesario para construir el sistema, y los costos operativos, vinculados a la infraestructura requerida para su funcionamiento. Conforme a los lineamientos del trabajo, no se realiza cálculo del retorno sobre la inversión. Los valores se expresan en pesos argentinos, tomando como referencia el tipo de cambio del dólar vendedor del Banco de la Nación Argentina al mes de junio de 2026, equivalente a un dólar estadounidense igual a \$1.460.

El desarrollo se planificó sobre un esquema de un único recurso con perfil de desarrollador full-stack junior, con dedicación completa durante los seis meses estimados para el proyecto, en coherencia con el cronograma de doce sprints presentado en el diseño metodológico. El honorario mensual de referencia se establece a partir del relevamiento del mercado de tecnología argentino para el año 2026, que ubica la remuneración de un desarrollador full-stack junior en el mercado local entre \$850.000 y \$1.550.000 mensuales; se adopta un valor intermedio de \$1.100.000. La Tabla 8 detalla el costo de desarrollo resultante.

Tabla 8: Costos de desarrollo

Rol	Tareas principales	Honorario mensual (\$)	Meses	Subtotal (\$)
Desarrollador Full-Stack Junior	Análisis y diseño de arquitectura, modelo de datos, desarrollo de backend (API) y frontend, integración de los modelos de aprendizaje automático, pruebas y despliegue.	1.100.000	6	6.600.000
Total de desarrollo (\$):				6.600.000

Fuente: elaboración propia con base en el relevamiento de remuneraciones del sector de tecnología en Argentina, 2026 (Glassdoor; encuestas salariales del sector) y en los honorarios orientativos del Consejo Profesional de Ciencias Informáticas.

Los costos operativos corresponden a la infraestructura necesaria para mantener el sistema en producción una vez desplegado. Una característica destacable del proyecto es que la totalidad de la pila tecnológica se sustenta en software de código abierto —FastAPI, PostgreSQL, React, scikit-learn, Prophet y ReportLab—, por lo que el costo de licencias de software es nulo. El único servicio externo de pago es la interfaz de inferencia de Groq, cuyo uso se encuentra acotado por diseño a la generación de resúmenes y recomendaciones sobre

datos agregados, lo que mantiene el consumo de tokens —y, por lo tanto, su costo— en valores marginales. La Tabla 9 resume los costos operativos mensuales estimados.

Tabla 9: Costos operativos mensuales

Concepto	Detalle	Costo mensual (\$)
Servidor VPS	Hetzner Cloud CX22 (2 vCPU, 4 GB RAM, 40 GB NVMe), €4,35/mes	6.900
Almacenamiento de respaldos	Almacenamiento de objetos para las copias de pg_dump, ≈ €3,20/mes	5.100
Dominio web	Dominio .com, ≈ USD 15/año (prorrateado)	1.825
API de inferencia (Groq)	Llama 3.3 70B; uso acotado a datos agregados (USD 0,59 / 0,79 por millón de tokens de entrada / salida)	7.300
Licencias de software	Pila de código abierto (FastAPI, PostgreSQL, React, scikit-learn, Prophet, ReportLab)	0
Total operativo mensual (\$):		21.125

Fuente: elaboración propia con base en los tarifarios públicos de Hetzner Cloud y de Groq, y en la cotización del Banco de la Nación Argentina (junio de 2026).

A los costos anteriores se suma, por única vez, el equipamiento de desarrollo: una computadora portátil de gama media, con un valor de referencia aproximado de USD 1.000 (alrededor de \$1.460.000), que puede encontrarse amortizada o ya disponible al inicio del proyecto. En síntesis, el costo de desarrollo asciende a \$6.600.000 para los seis meses de trabajo, mientras que la operación del sistema demanda aproximadamente \$21.125 mensuales. La estructura de costos confirma que la decisión de procesamiento local y de adopción de tecnologías abiertas no solo responde a la política de soberanía de datos, sino que además reduce de manera significativa los costos recurrentes frente a alternativas basadas en servicios comerciales de pago.

Análisis de Riesgos

El análisis de riesgos identifica los principales factores que podrían comprometer el desarrollo o el funcionamiento del proyecto, ponderando para cada uno su probabilidad de ocurrencia (Alta, Intermedia o Baja) y su impacto en una escala de 1, bajo, a 5, muy alto. Con el fin de priorizar el tratamiento, se calcula la exposición al riesgo como el producto de ambas variables —asignando los valores 3, 2 y 1 a las probabilidades Alta, Intermedia y Baja— y se ordena la tabla de mayor a menor exposición. Este ordenamiento responde al Principio de Pareto: un grupo reducido de riesgos concentra la mayor parte de la amenaza potencial sobre el proyecto y, por lo tanto, merece la atención prioritaria.

Tabla 10: Matriz de riesgos del proyecto

ID	Tipo	Riesgo	Prob.	Impacto	Exp.	Acciones de contingencia
1	Técnico	Historial financiero insuficiente para que el modelo Prophet genere proyecciones confiables.	Alta	3	9	Estrategia de arranque en frío con media móvil; aviso al usuario sobre el carácter referencial de la proyección hasta acumular datos.
2	Negocio	Cambios en la normativa del Monotributo (escalas y categorías de ARCA) que desactualicen el módulo fiscal.	Alta	3	9	Catálogo de categorías versionado en base de datos, actualizable por carga de datos sin modificar el código.
3	Proyecto	Sub o sobreestimación de los tiempos de desarrollo en un proyecto unipersonal.	Alta	3	9	Gestión con Scrum en sprints de dos semanas, alcance acotado a un producto mínimo viable y revisión continua del avance.

4	Proyecto	Dependencia de un único desarrollador (bus factor), que concentra el conocimiento del sistema.	Intermedia	4	8	Documentación técnica del proyecto, código versionado en repositorio remoto y suite de pruebas automatizadas.
5	Técnico	Indisponibilidad o cambios en el servicio externo de Groq, que afectan resúmenes y recomendaciones.	Intermedia	3	6	Mecanismo de reserva local con plantillas predefinidas, ya implementado; el resto del sistema opera con independencia del servicio.
6	Técnico	Fuga o exposición de datos financieros sensibles del usuario.	Baja	5	5	Hash de contraseñas con bcrypt, autenticación por token, aislamiento por usuario, política de soberanía de datos y copias de respaldo con acceso restringido.
7	Técnico	Pérdida de datos por falla del servidor o de la base de datos.	Baja	5	5	Política de respaldo automatizada (pg_dump diario con réplica en la nube) y restauraciones de prueba periódicas.
8	Técnico	Precisión insuficiente del clasificador en categorías amplias que se solapan (Marketing, Servicios).	Intermedia	2	4	Reentrenamiento del modelo a partir de las correcciones del usuario y ampliación progresiva del conjunto de entrenamiento.
9	Técnico	Errores en la importación de extractos bancarios por formatos no contemplados.	Intermedia	2	4	Diccionario de sinónimos de columnas ampliable, validación previa del archivo y persistencia transaccional con reversión ante errores.

Fuente: elaboración propia. La exposición surge del producto entre probabilidad (Alta = 3, Intermedia = 2, Baja = 1) e impacto (1 a 5).

Del ordenamiento se desprende que los riesgos de mayor exposición se concentran en cuatro factores: la disponibilidad de datos para la proyección, la actualización de la normativa fiscal, la estimación de tiempos y la dependencia de un único desarrollador. Sobre ellos se enfoca el esfuerzo de mitigación, en línea con el Principio de Pareto. Es relevante señalar que varias de las acciones de contingencia descritas no son meramente previstas, sino que ya forman parte del sistema desarrollado —el arranque en frío del motor de predicción, el catálogo fiscal versionado, el mecanismo de reserva ante la caída del servicio externo y la persistencia transaccional de la importación—, lo que reduce de manera efectiva la exposición de los riesgos técnicos asociados.

Conclusiones

El presente trabajo final de graduación consistió en el desarrollo de FreelanceControl, un sistema de gestión financiera predictiva para monotributistas argentinos. La elección del tema nació de una motivación personal y concreta: anticipar una problemática que yo mismo enfrentaría como profesional independiente. El trabajo freelance no ofrece ingresos estables ni fijos, y advertí que las aplicaciones bancarias actuales no resuelven esa necesidad de tener, en pocos clics, una mirada clara sobre la propia economía y su evolución mes a mes. A partir de esa observación me propuse construir una herramienta que se ajustara a mis necesidades reales y a las de quienes comparten esta forma de trabajo: consolidar los movimientos, proyectar los ingresos futuros y, sobre esa base, poder decidir con anticipación cuándo ahorrar, cuándo invertir y qué medidas tomar frente a lo que pudiera ocurrir.

Los objetivos planteados al inicio se cumplieron. El sistema integra un clasificador de gastos por procesamiento de lenguaje natural entrenado localmente, que alcanzó una exactitud del 76 % sobre doce categorías, por encima del umbral del 70 % fijado como meta. Se implementó el motor de predicción de ingresos a seis meses con Prophet y su estrategia de arranque en frío, el módulo fiscal con monitoreo de la categoría de Monotributo y el componente de auditoría automatizada, todo bajo una arquitectura que preserva la soberanía sobre los datos del usuario. De este modo, la propuesta de solución respondió a cada una de las causas identificadas en el diagnóstico.

Desde lo profesional, el proyecto me permitió aplicar conocimientos adquiridos a lo largo de la carrera, en particular los referidos al análisis y diseño de la arquitectura del sistema y al modelado de bases de datos. Al mismo tiempo, me exigió aprender desde cero el desarrollo frontend, la lógica de los modelos de aprendizaje automático, el consumo de APIs, el versionado con Git y el uso de contenedores en Docker. El mayor desafío técnico estuvo en los modelos de machine learning y en la importación de movimientos bancarios, dos componentes que demandaron iteración y estudio. Sumo a ello la incorporación del uso de inteligencia artificial como herramienta de trabajo, a través de formación específica, en un contexto donde su avance resulta imposible de ignorar y su aplicación correcta se vuelve cada vez más necesaria en la profesión.

En lo personal, haber llevado el proyecto hasta el final me deja una enorme satisfacción y la certeza de haber cumplido lo que me propuse a comienzos de año. Cada traba y cada problema se transformaron en un desafío que me obligó a buscar soluciones, investigar e incorporar nuevas tecnologías. Más allá del resultado, ese proceso —el de enfrentar lo desconocido y resolverlo— es lo que más valoro de esta experiencia.

Anexos

A continuación se incorpora documentación complementaria que respalda los resultados expuestos en el cuerpo del trabajo.

Anexo A. Métricas del clasificador de gastos. Las siguientes figuras presentan la evaluación del modelo base mediante validación cruzada de cinco particiones sobre un conjunto de seiscientos ejemplos. En esta técnica, cada ejemplo es predicho cuando no formó parte del bloque de entrenamiento, lo que ofrece una medida honesta del desempeño. La exactitud global obtenida es del 76 %, por encima de la meta del 70 %. La evaluación es reproducible ejecutando el script de evaluación incluido en el proyecto.

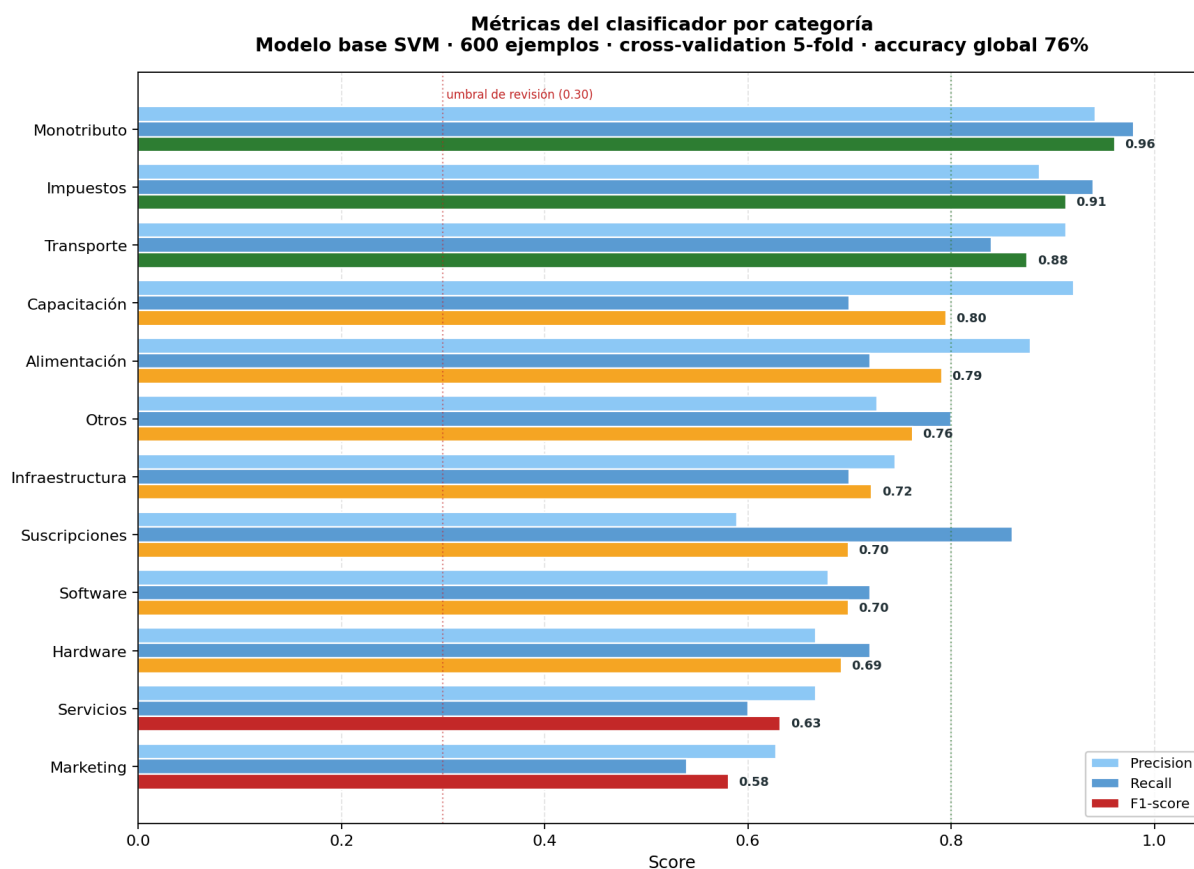


Figura 6: Precisión, exhaustividad y F1 por categoría del clasificador de gastos.

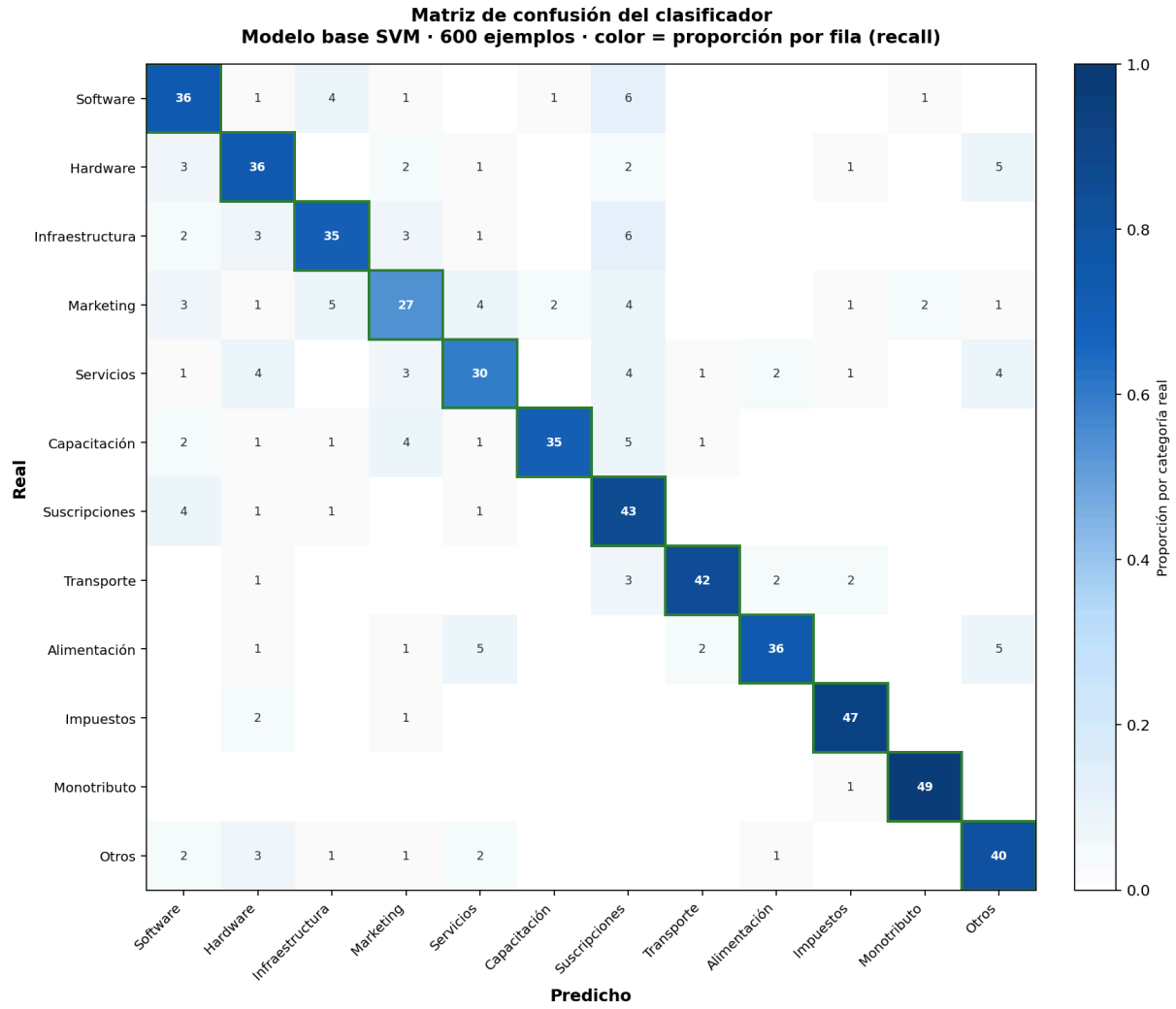


Figura 7: Matriz de confusión del clasificador (validación cruzada de cinco particiones).

Resumen

FreelanceControl es un sistema web de gestión financiera predictiva orientado a los trabajadores independientes inscriptos en el régimen de Monotributo en Argentina. El proyecto surge de una problemática concreta del segmento: ingresos irregulares, riesgo de recategorización fiscal y una marcada dispersión de la información entre planillas, comprobantes y aplicaciones bancarias. Para responder a esas necesidades, el sistema consolida ingresos, gastos y facturas bajo un único modelo de datos por usuario e incorpora cuatro componentes automatizados: un clasificador de gastos basado en procesamiento de lenguaje natural entrenado localmente, que alcanzó una exactitud del 76 % sobre doce categorías; un motor de predicción de ingresos a seis meses mediante el modelo Prophet, con una estrategia de arranque en frío para usuarios con poco historial; un módulo fiscal que monitorea el porcentaje de utilización del límite de la categoría de Monotributo; y un componente de auditoría que detecta gastos duplicados, anomalías estadísticas y facturas vencidas. La arquitectura, construida sobre FastAPI, PostgreSQL y React, prioriza la soberanía de los datos: el procesamiento sensible se ejecuta de forma local y el uso de servicios externos se restringe a información numérica agregada. Los resultados confirman que la solución cumple los objetivos planteados y ofrece al monotributista una herramienta accesible para anticipar su situación financiera y tomar decisiones informadas.

Palabras clave: Monotributo, gestión financiera, aprendizaje automático, predicción de ingresos, soberanía de datos.

Abstract

FreelanceControl is a predictive financial management web system aimed at self-employed workers registered under Argentina's Monotributo simplified tax regime. The project addresses a concrete problem within this segment: irregular income, the risk of tax recategorization, and the scattering of information across spreadsheets, receipts, and banking applications. To meet these needs, the system consolidates income, expenses, and invoices under a single per-user data model and incorporates four automated components: an expense classifier based on natural language processing trained locally, which reached 76% accuracy across twelve categories; an income forecasting engine over a six-month horizon using the Prophet model, with a cold-start strategy for users with little history; a tax module that monitors the usage percentage of the Monotributo category limit; and an auditing component that detects duplicate expenses, statistical anomalies, and overdue invoices. The architecture, built on FastAPI, PostgreSQL, and React, prioritizes data sovereignty: sensitive processing runs locally and the use of external services is restricted to aggregated numerical information. The results confirm that the solution meets the proposed objectives and provides self-employed workers with an accessible tool to anticipate their financial situation and make informed decisions.

Keywords: Monotributo, financial management, machine learning, income forecasting, data sovereignty.

Referencias

- Administración Federal de Ingresos Públicos. (2024). *Régimen Simplificado para Pequeños Contribuyentes. Monotributo*. Recuperado de <https://www.afip.gob.ar>
- Banco de la Nación Argentina. (2026). *Cotización de monedas*. Recuperado de <https://www.bna.com.ar>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.
- Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley Professional.
- Consejo Profesional de Ciencias Informáticas de la Provincia de Córdoba. (2026). *Honorarios orientativos para profesionales de la informática*. Recuperado de <https://www.cpcipc.org.ar>
- Glassdoor. (2026). *Sueldo: Desarrollador Full Stack Junior en Argentina*. Recuperado de <https://www.glassdoor.com.ar>
- Groq, Inc. (2026). *Pricing: Groq API for large language models*. Recuperado de <https://groq.com/pricing>
- Hetzner Online GmbH. (2026). *Hetzner Cloud — Shared vCPU servers (CX line)*. Recuperado de <https://www.hetzner.com/cloud>
- Ley N° 25.326. (2000). *Ley de Protección de los Datos Personales*. Boletín Oficial de la República Argentina. Buenos Aires, Argentina, 4 de octubre de 2000.
- López, R., y Fernández, M. (2020). *Gestión financiera para profesionales independientes*. Editorial Ediciones Académicas.
- Martínez, J. (2019). Trabajo autónomo en la era digital: nuevas formas de empleo y desafíos administrativos. *Revista de Estudios del Trabajo*, 25(3), 45-67.
- Medina Cruz, A., Pinedo Ballesteros, P. C., y Téllez Acuña, F. R. (2019). *Metodologías para el desarrollo de software seguro*. Editorial Universidad Distrital.
- Momjian, B. (2022). *PostgreSQL: introduction and concepts*. Addison-Wesley Professional.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,

- Brucher, M., Perrot, M., y Duchesnay, E. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Ramírez, S. (2021). *FastAPI: modern Python web development*. O'Reilly Media.
- Rumbaugh, J., Jacobson, I., y Booch, G. (2011). *UML: Lenguaje unificado de modelado* (3.^a ed.). Pearson Educación.
- Schwaber, K., y Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum*. Scrum.org.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J. F., y Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503-2511.
- Taylor, S. J., y Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37-45.